

**Software Design Principles  
(Chapter 3) from  
Dive Into DESIGN PATTERNS**

# Encapsulation on Method Level (before)

```
1  method getOrderTotal(order) is  
2      total = 0  
3      foreach item in order.lineItems  
4          total += item.price * item.quantity  
5  
6      if (order.country == "US")  
7          total += total * 0.07 // US sales tax  
8      else if (order.country == "EU"):  
9          total += total * 0.20 // European VAT  
10  
11     return total
```

# Encapsulation on Method Level (after)

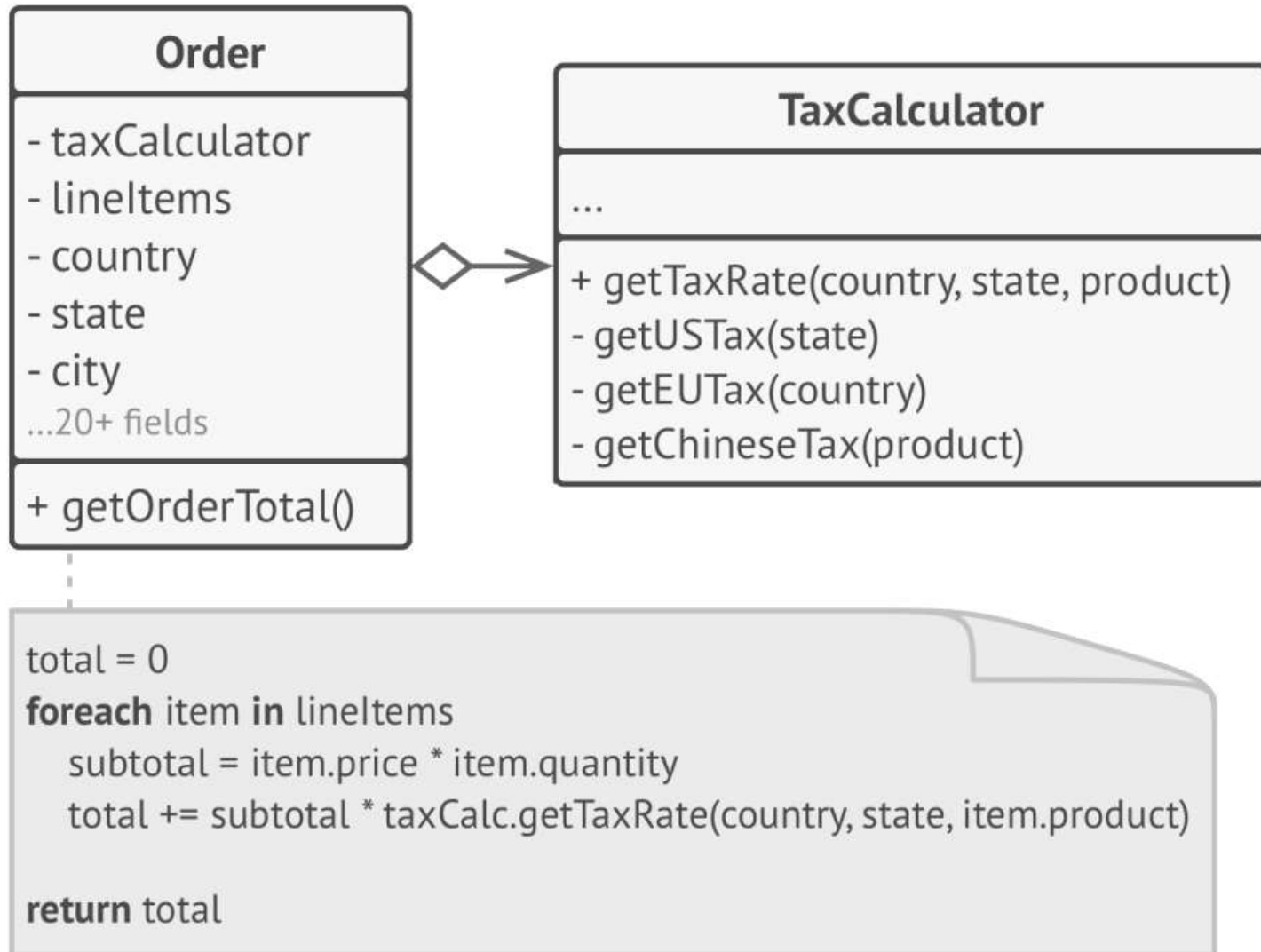
```
1  method getOrderTotal(order) is
2      total = 0
3      foreach item in order.lineItems
4          total += item.price * item.quantity
5
6      total += total * getTaxRate(order.country)
7
8      return total
9
10 method getTaxRate(country) is
11     if (country == "US")
12         return 0.07 // US sales tax
13     else if (country == "EU")
14         return 0.20 // European VAT
15     else
16         return 0
```

# Encapsulation: class level (before)



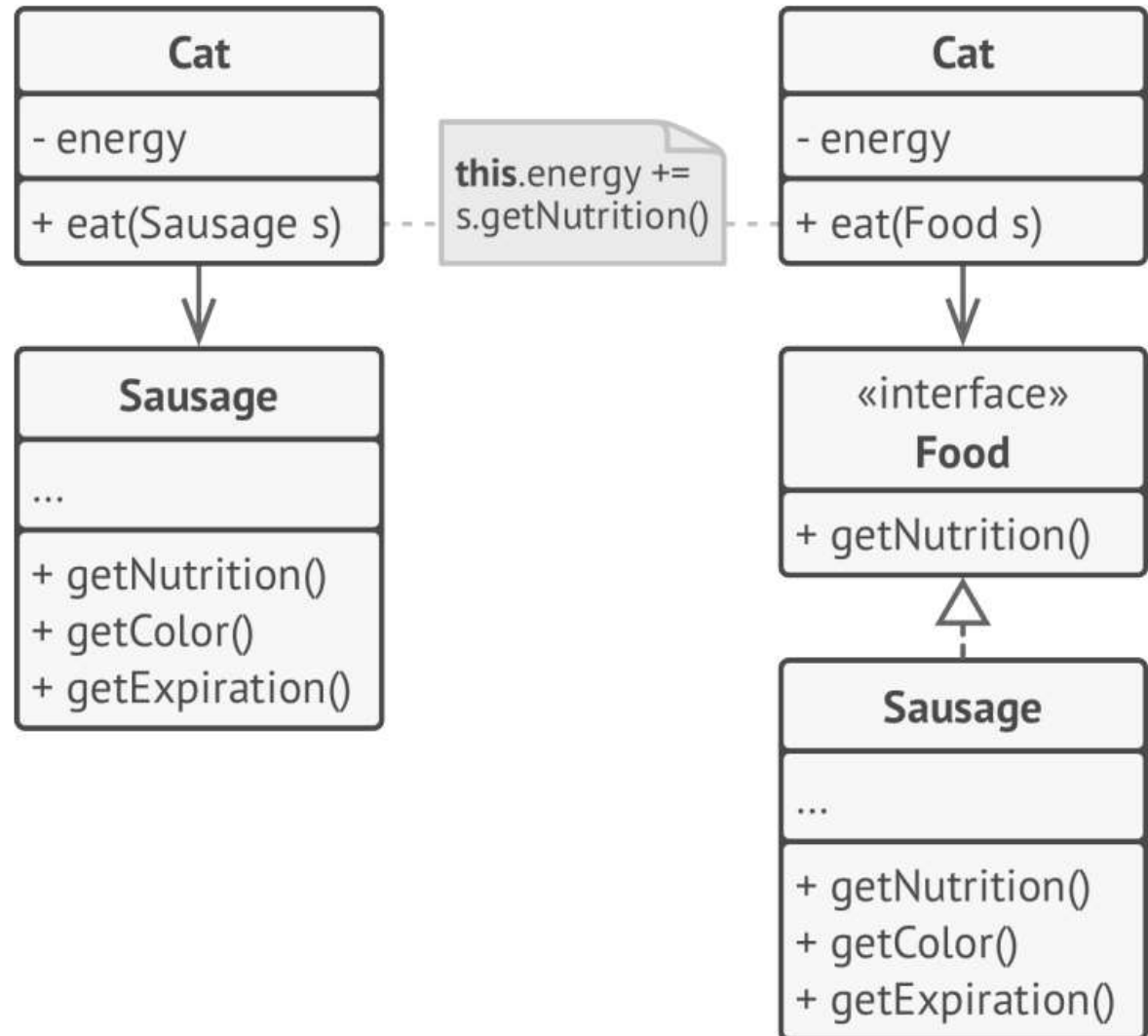
*BEFORE: calculating tax in `Order` class.*

# Encapsulation: class level (before)

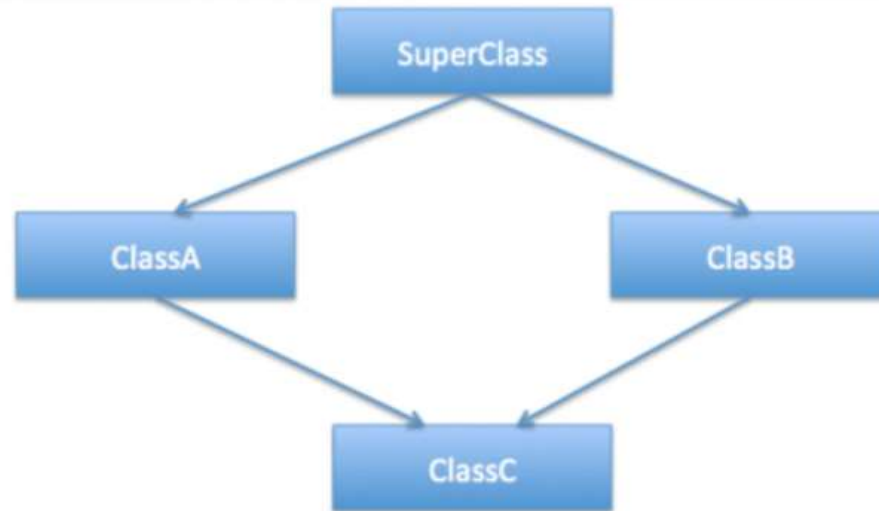


# Interface is Better than Implementation

After making this change, you won't probably feel any immediate benefit. On the contrary, the code has become more complicated than it was before. However, if you feel that this might be a good extension point for some extra functionality, or that some other people who use your code might want to extend it here, then go for it.



# Diamond Problem in Multiple Inheritance

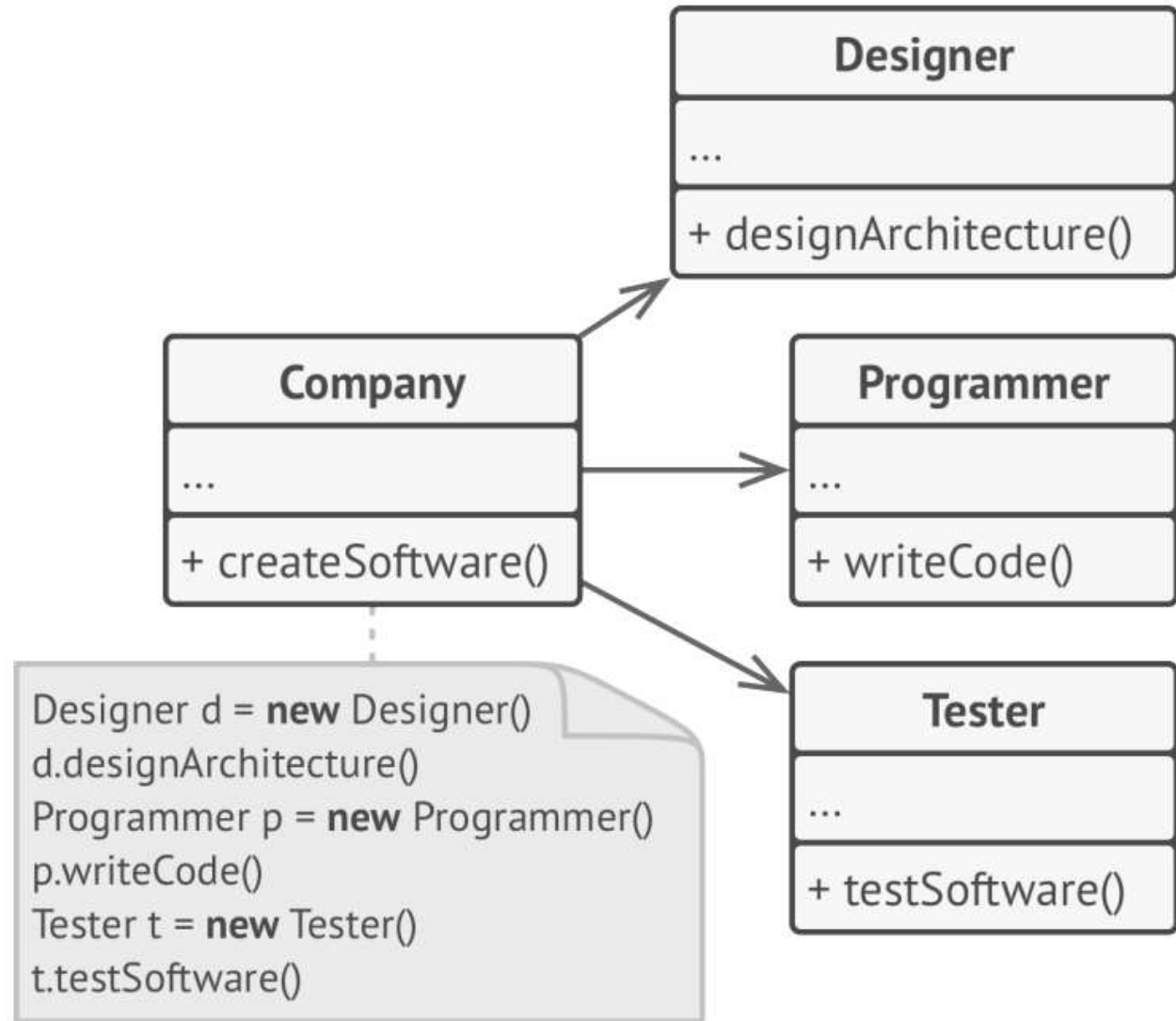


Both Class A and Class B have implementation of doSomething()

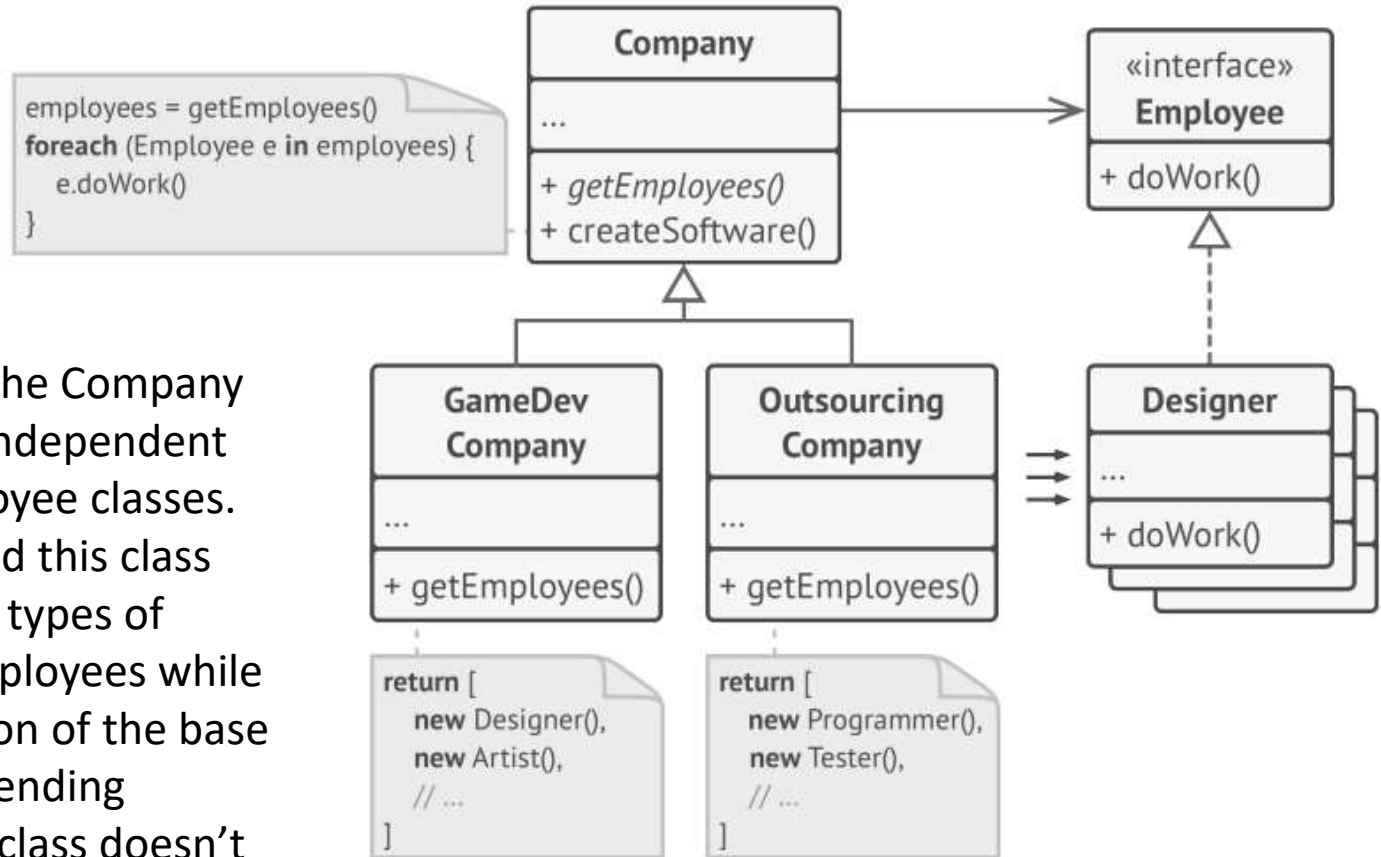
```
// this is just an assumption to explain the diamond problem
//this code won't compile
public class ClassC extends ClassA, ClassB{

    public void test(){
        //calling super class method
        doSomething();
    }
}
```

# Example of Tight Coupling



# Getting rid of tight coupling



After this change, the **Company** class has become independent from various employee classes. Now you can extend this class and introduce new types of companies and employees while still reusing a portion of the base company class. Extending the base company class doesn't break any existing code that already relies on it.

# SOLID Design Principles

**S**ingle responsibility principle

**O**pen / closed principle

**L**iskov substitution principle

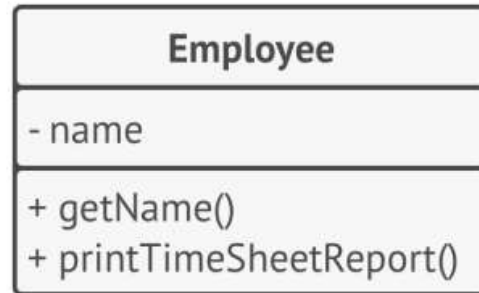
**I**nterface segregation principle

**D**ependency inversion principle

- *SOLID* is a mnemonic for five design principles intended to make software designs more understandable, flexible and maintainable
- The cost of applying these principles into a program's architecture might be making it more complicated than it should be.

# Single Responsibility Principle: A class should have just one reason to change.

*Every class should have a single responsibility, and that responsibility should be entirely encapsulated by the class.*

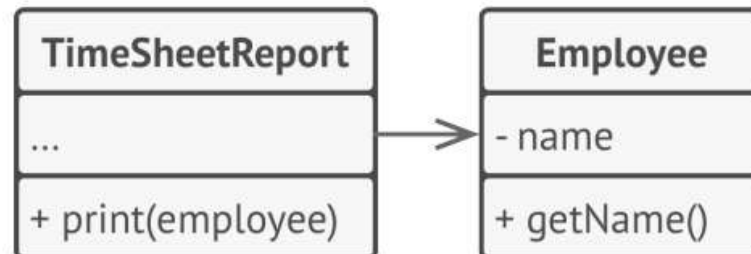


*BEFORE: the class contains several different behaviors.*

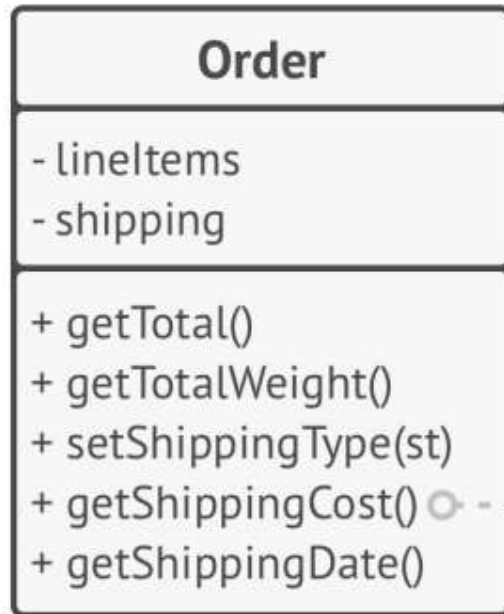
- ✓ Less fragile code
- ✓ Low coupling
- ✓ High cohesion

- Changeability
- Maintainability
- Testability and easier debugging

Solve the problem by moving the behavior related to printing timesheet reports into a separate class. This change lets you move other report-related stuff to the new class.



# Open/Closed Principle: Classes should be open for extension but closed for modification

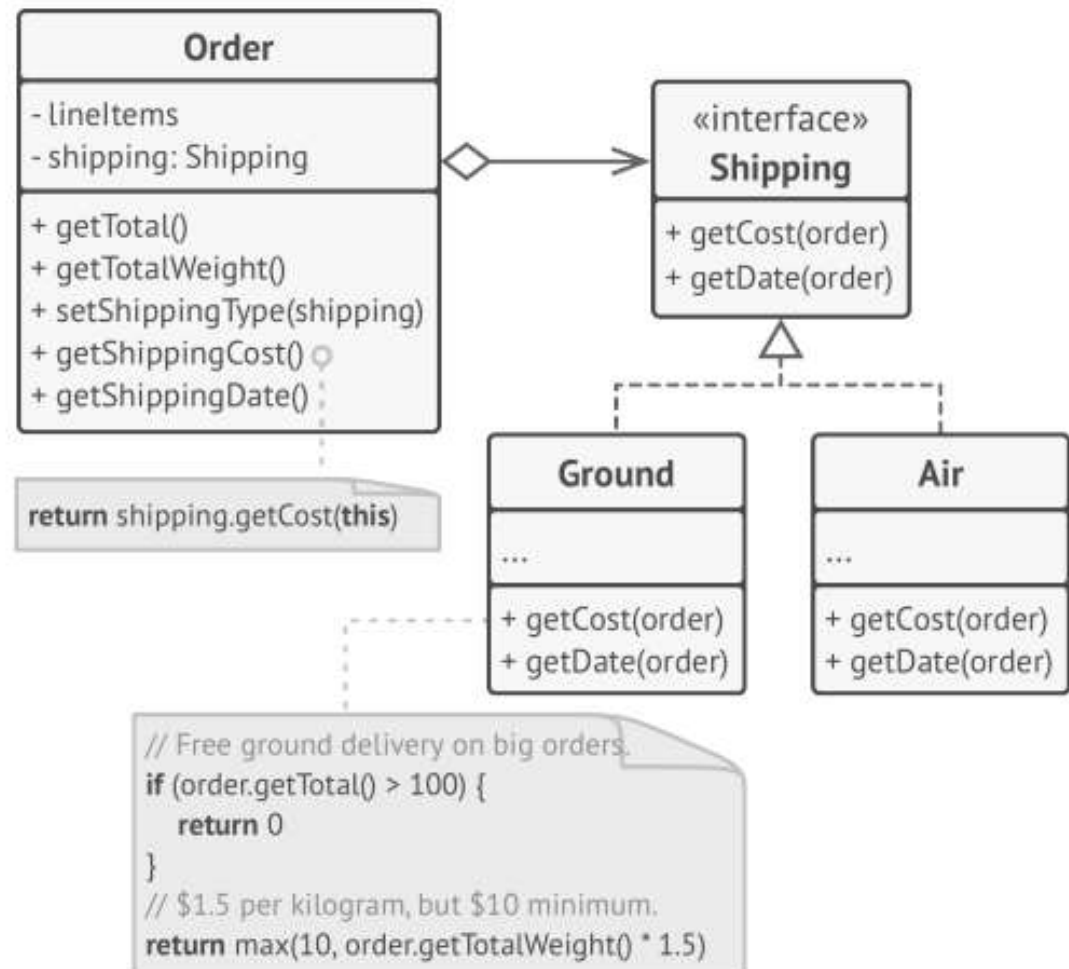


```
if (shipping == "ground") {  
    // Free ground delivery on big orders.  
    if (getTotal() > 100) {  
        return 0  
    }  
    // $1.5 per kilogram, but $10 minimum.  
    return max(10, getTotalWeight() * 1.5)  
}  
  
if (shipping == "air") {  
    // $3 per kilogram, but $20 minimum.  
    return max(20, getTotalWeight() * 3)  
}
```

*BEFORE: you have to change the **Order** class whenever you add a new shipping method to the app.*

You can solve the problem by applying the *Strategy* pattern. Start by extracting shipping methods into separate classes with a common interface.

The idea was that once completed, the implementation of a class could **only be modified** to correct errors; new or changed features would require that a **different class be created**.



```
public class Rectangle
{
    public double Width { get; set; }
    public double Height { get; set; }
}

public class AreaCalculator
{
    public double Area(Rectangle[] shapes)
    {
        double area = 0;
        foreach (var shape in shapes)
        {
            area += shape.Width*shape.Height;
        }

        return area;
    }
}
```

# Add Circle

```
public double Area(object[] shapes)
{
    double area = 0;
    foreach (var shape in shapes)
    {
        if (shape is Rectangle)
        {
            Rectangle rectangle = (Rectangle) shape;
            area += rectangle.Width*rectangle.Height;
        }
        else
        {
            Circle circle = (Circle)shape;
            area += circle.Radius * circle.Radius * Math.PI;
        }
    }

    return area;
}
```

# Add Ellipse, Triangle, ...?

```
public abstract class Shape
{
    public abstract double Area();
}

public class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }
    public override double Area()
    {
        return Width*Height;
    }
}

public class Circle : Shape
{
    public double Radius { get; set; }
    public override double Area()
    {
        return Radius*Radius*Math.PI;
    }
}
```

# Add Ellipse in OCP

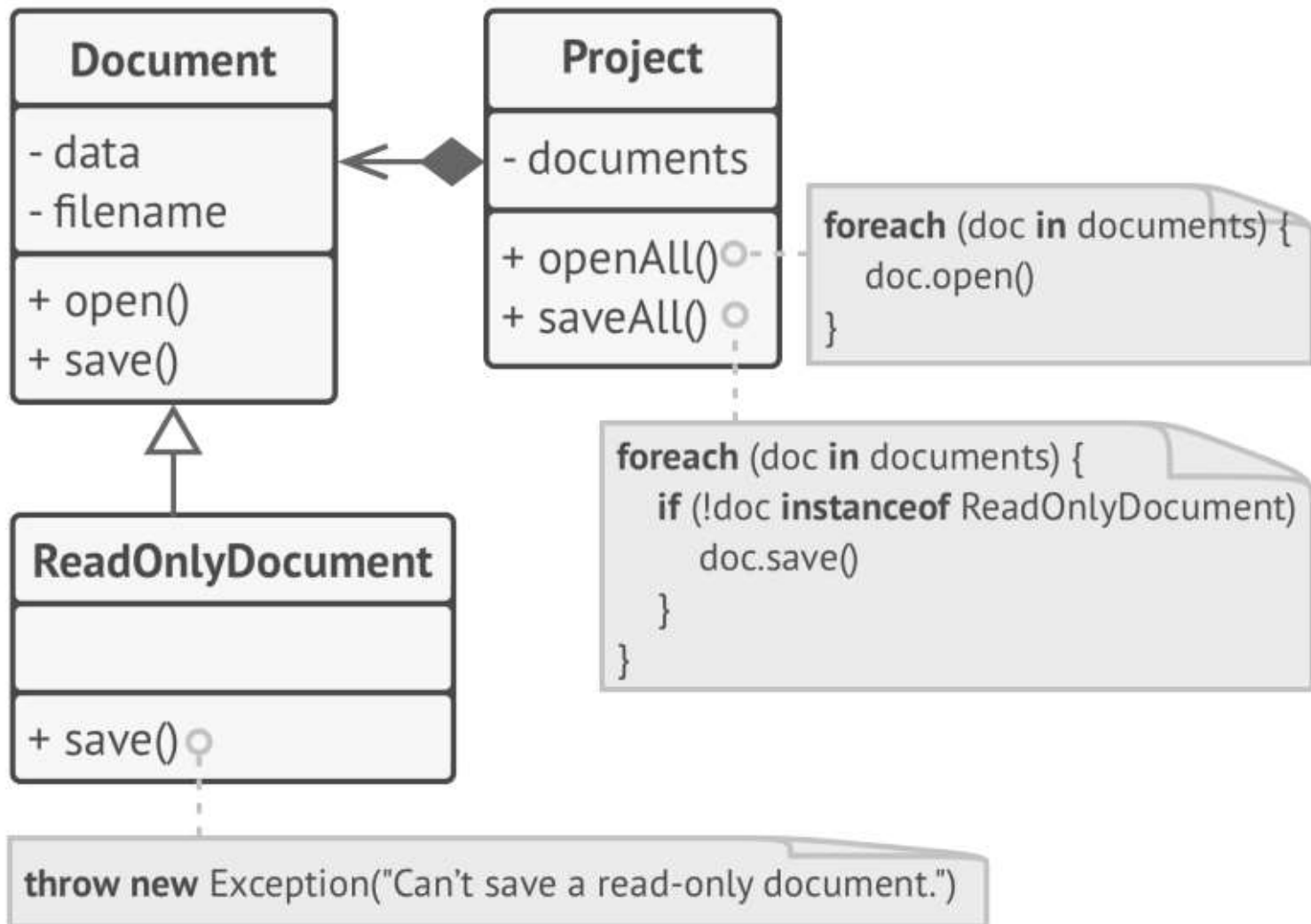
```
public class AreaCalculator
{
    public double Area(Shape[] shapes)
    {
        double area = 0;
        foreach (Shape shape in shapes)
        {
            area += shape.Area();
        }
        return area;
    }
}

public class Ellipse : Shape
{
    public double majorRadius { get; set; }
    public double minorRadius { get; set; }

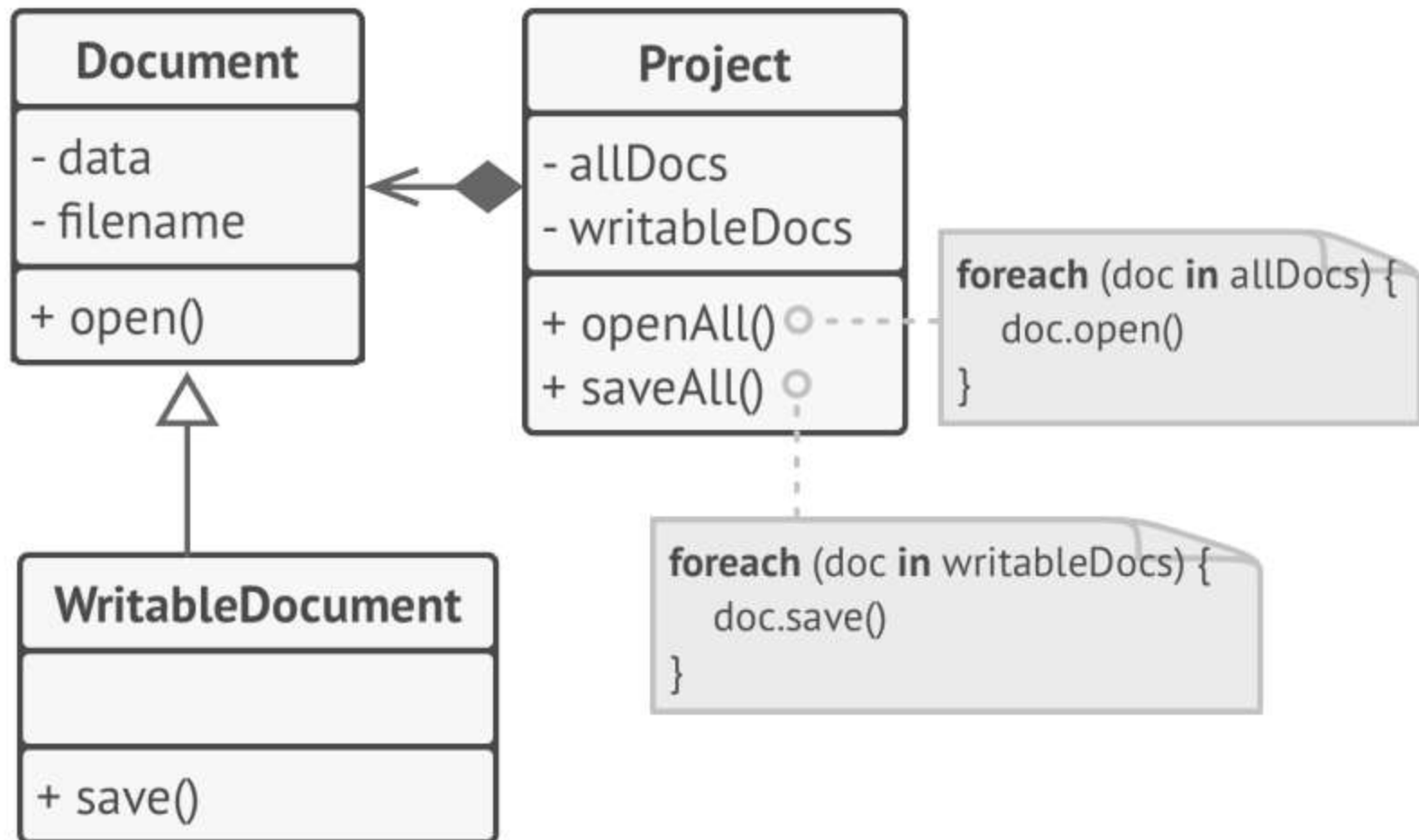
    public override double Area()
    {
        return majorRadius * minorRadius * Math.PI;
    }
}
```

# Liskov Substitution Principle

- When extending a class, remember that you should be able to pass objects of the subclass in place of objects of the parent class without breaking the client code.
- This means that the subclass should remain compatible with the behavior of the superclass.
- In other words, every subclass/derived class should be substitutable for their base/parent class.
- When overriding a method, extend the base behavior rather than replacing it with something else entirely.
- The substitution principle is a set of checks that help predict whether a subclass remains compatible with the code that was able to work with objects of the superclass.



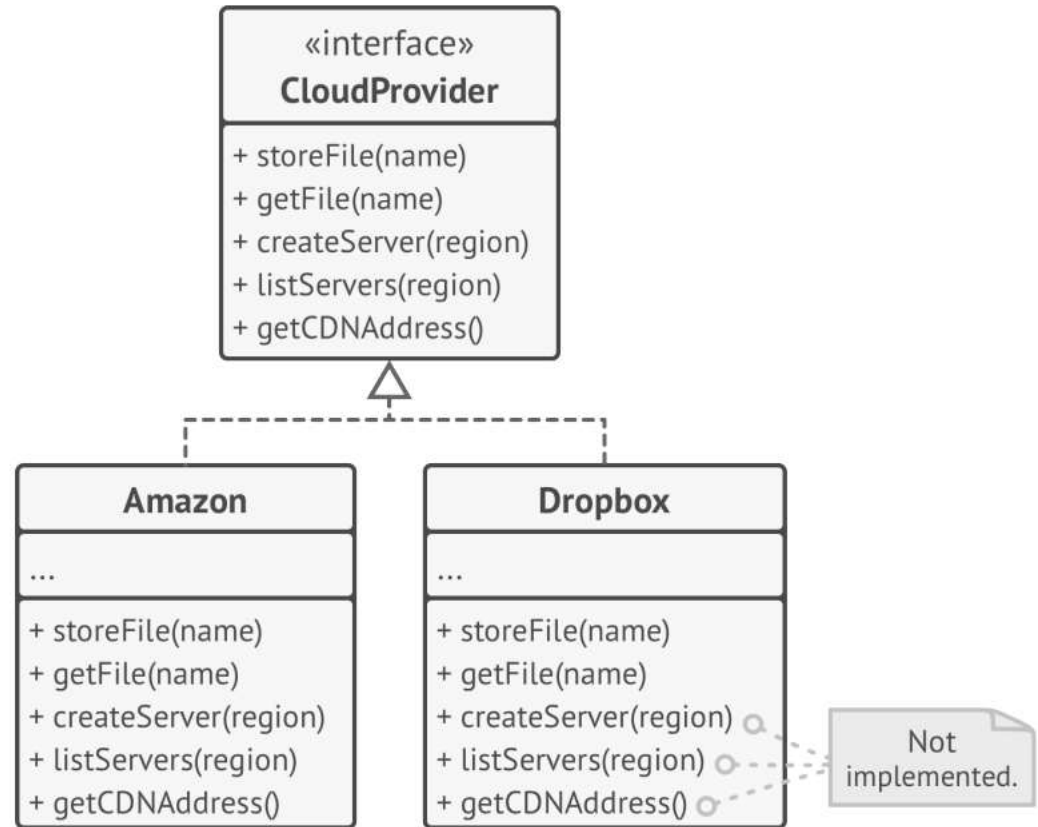
*BEFORE: saving doesn't make sense in a read-only document, so the subclass tries to solve it by resetting the base behavior in the overridden method.*



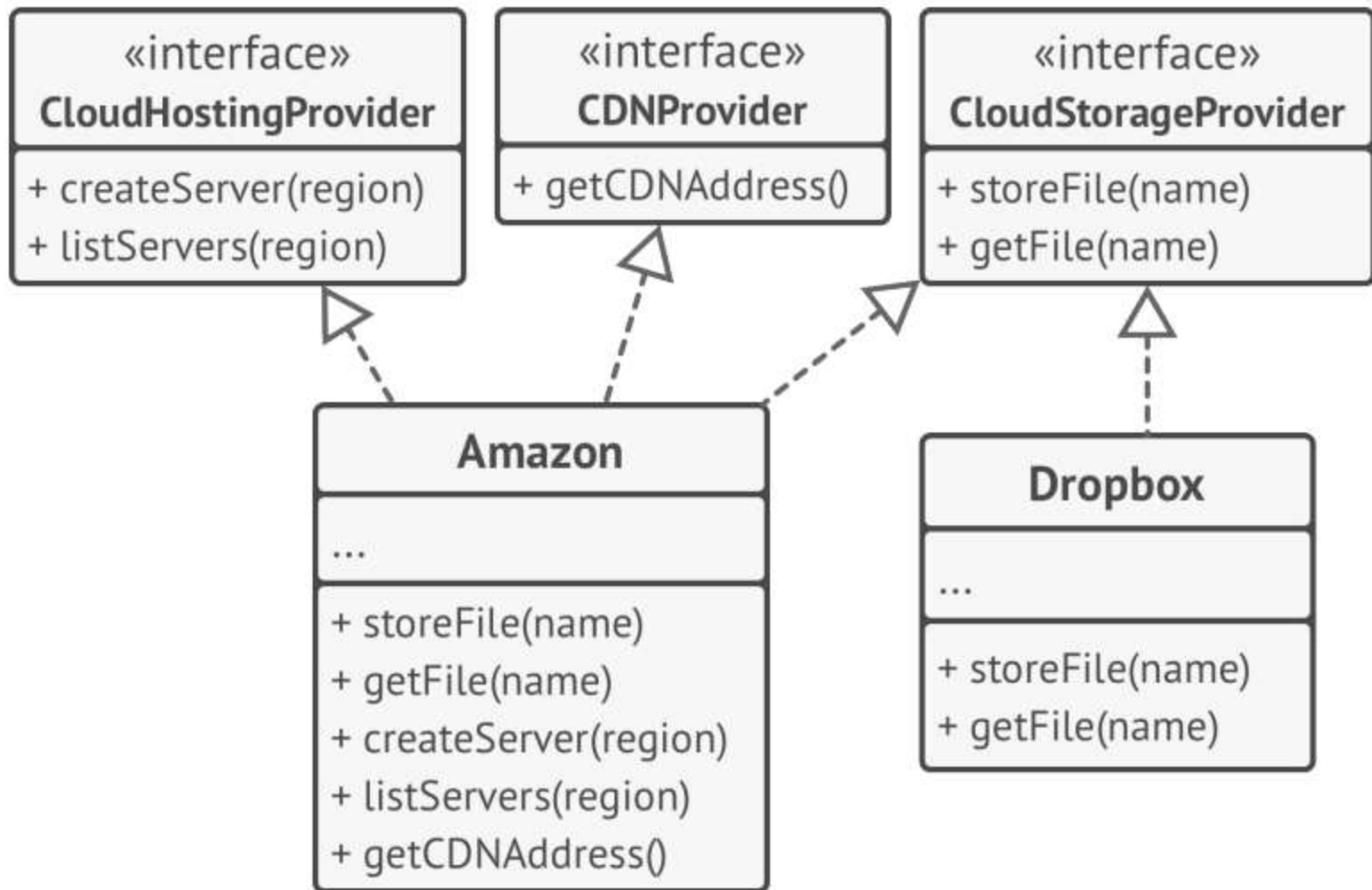
*AFTER: the problem is solved after making the read-only document class the base class of the hierarchy.*

# Interface Segregation Principle

- Clients shouldn't be forced to depend on methods they do not use.
- Try to make your interfaces narrow enough that client classes don't have to implement behaviors they don't need.



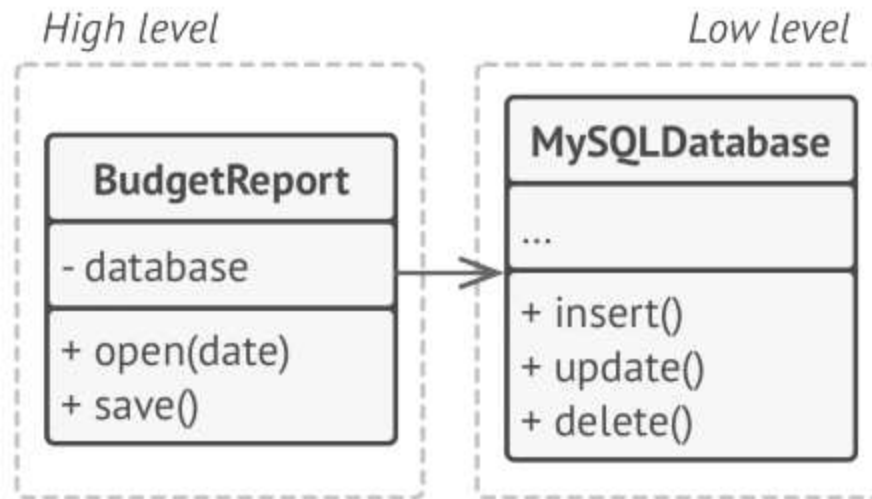
*BEFORE: not all clients can satisfy the requirements of the bloated interface.*



*AFTER: one bloated interface is broken down into a set of more granular interfaces.*

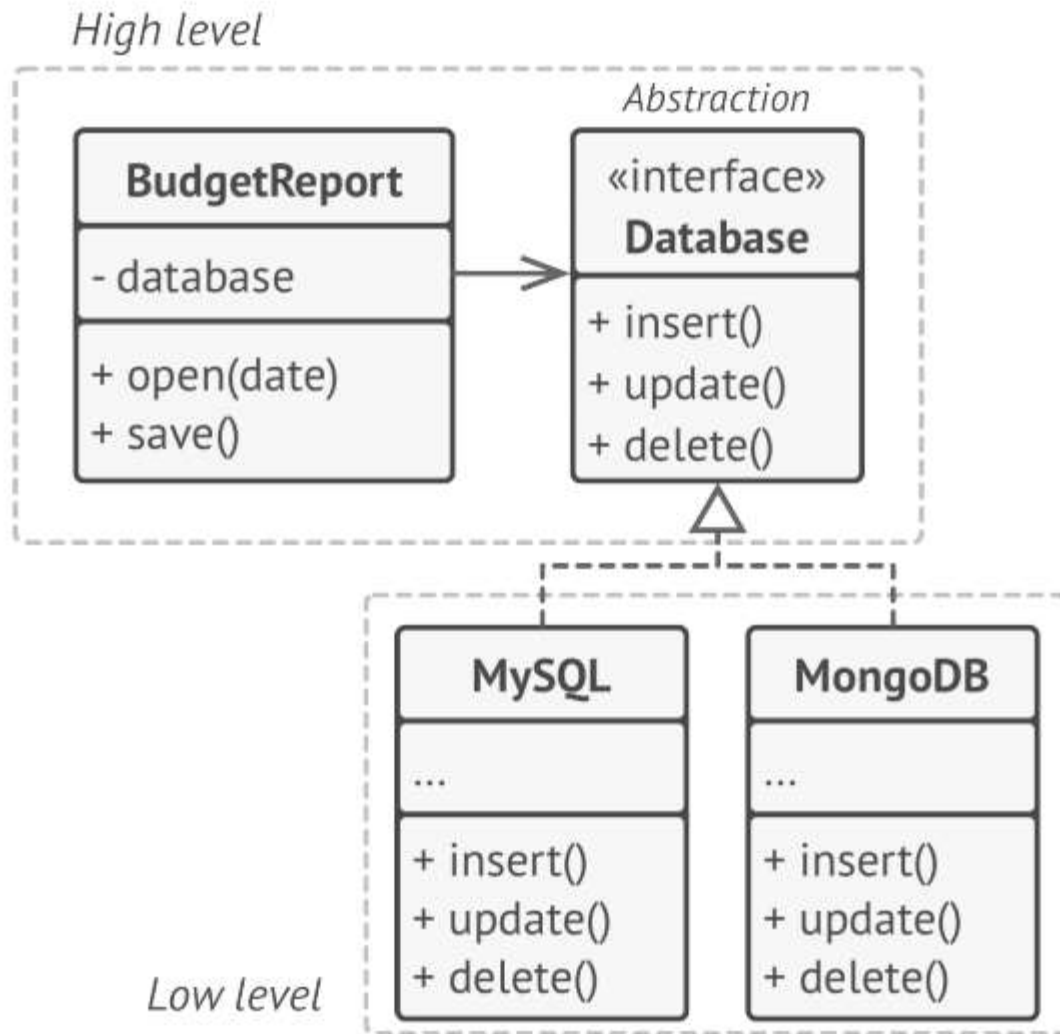
# Dependency Inversion Principle

- High-level classes shouldn't depend on low-level classes. Both should depend on abstractions.
- Abstractions shouldn't depend on details. Details should depend on abstractions.



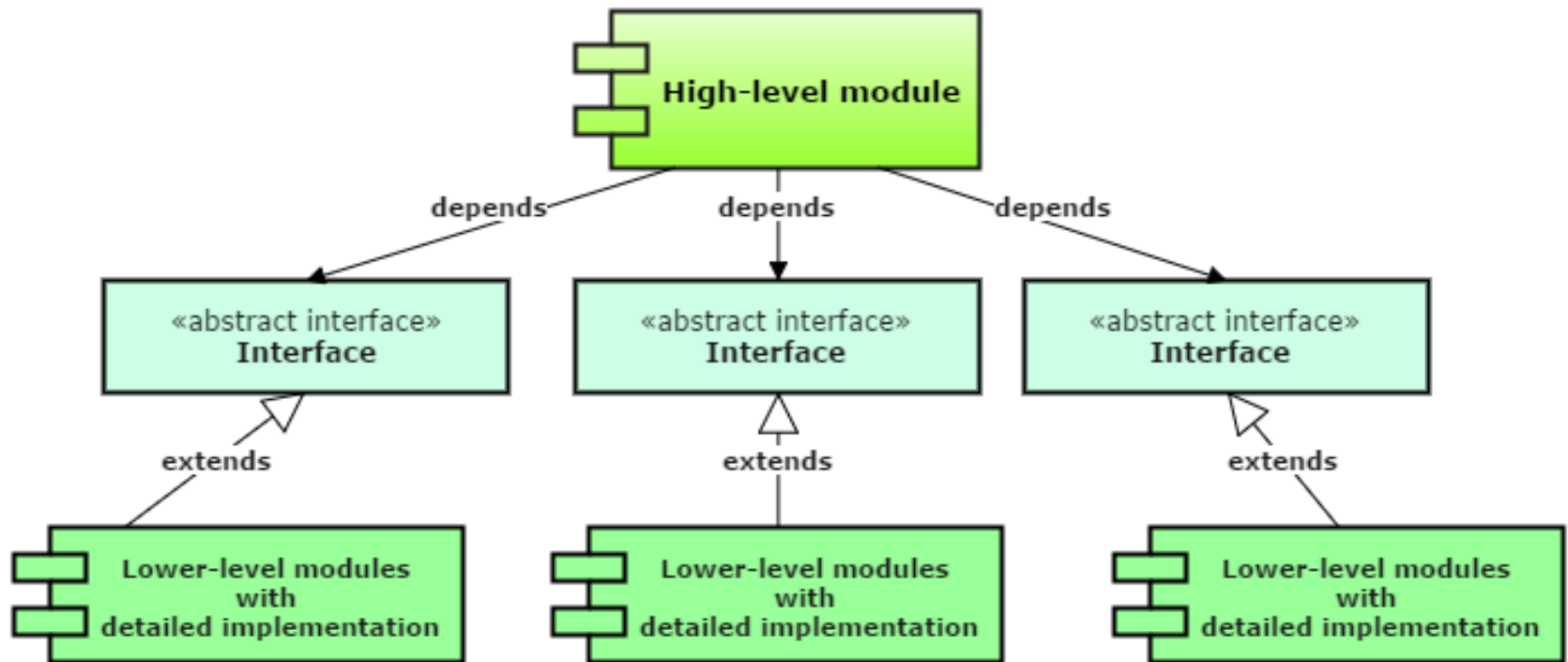
*BEFORE: a high-level class depends on a low-level class.*

**Depend upon Abstractions. Do not depend upon concretions.**



*AFTER: low-level classes depend on a high-level abstraction.*

# Inverted Dependency



# How DIP works?

- The High-level module does not depend on concrete 2nd level modules. Instead, the High-level module depends on abstract interfaces which are defined based on the services the High-level module requires from the Lower-level modules.
- Secondly, the lower-level modules extend the abstract interfaces. So, when the High-level module invokes the abstract interface, the required task is actually serviced by one of the Lower-level module implementations.
- Thus, both the High-level module and the Lower-level modules are concerned with the abstract interface structure and depend on it.

# Example: Without DIP

```
public class LightBulb {  
    public void turnOn() {  
        System.out.println("LightBulb: Bulb turned on...");  
    }  
    public void turnOff() {  
        System.out.println("LightBulb: Bulb turned off...");  
    }  
}
```

# Example: Without DIP

```
public class ElectricPowerSwitch {  
    public LightBulb lightBulb;  
    public boolean on;  
    public ElectricPowerSwitch(LightBulb lightBulb) {  
        this.lightBulb = lightBulb;  
        this.on = false;  
    }  
    public boolean isOn() {  
        return this.on;  
    }  
    public void press(){  
        boolean checkOn = isOn();  
        if (checkOn) {  
            lightBulb.turnOff();  
            this.on = false;  
        } else {  
            lightBulb.turnOn();  
            this.on = true;  
        }  
    }  
}
```

# Example: DIP

```
public interface Switch {  
    boolean isOn();  
    void press();  
}
```

```
public interface Switchable {  
    void turnOn();  
    void turnOff();  
}
```

# Example: DIP

```
public class ElectricPowerSwitch implements Switch {
    public Switchable client;
    public boolean on;
    public ElectricPowerSwitch(Switchable client) {
        this.client = client;
        this.on = false;
    }
    public boolean isOn() {
        return this.on;
    }
    public void press(){
        boolean checkOn = isOn();
        if (checkOn) {
            client.turnOff();
            this.on = false;
        } else {
            client.turnOn();
            this.on = true;
        }
    }
}
```

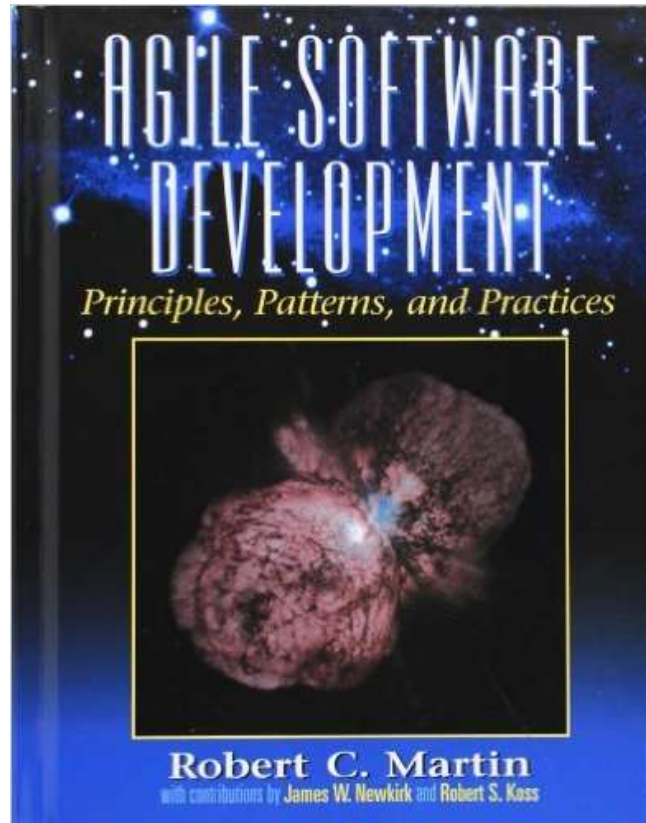
# Example: DIP

```
public class Fan implements Switchable {
    @Override
    public void turnOn() {
        System.out.println("Fan: Fan turned on...");
    }
    @Override
    public void turnOff() {
        System.out.println("Fan: Fan turned off...");
    }
}

public class LightBulb implements Switchable {
    @Override
    public void turnOn() {
        System.out.println("LightBulb: Bulb turned on...");
    }

    @Override
    public void turnOff() {
        System.out.println("LightBulb: Bulb turned off...");
    }
}
```

# Reference



**Agile Software Development, Principles, Patterns, and Practices  
by Robert C. Martin**